TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
ENERO -MAYO 2026
CARRERA:

Ingeniería en Sistemas Computacionales

MATERIA:
temas avanzados de desarrollo
nombre práctica:
Actividad de evidencia Individual del patrón: Prototipo

UNIDAD A EVALUAR:
Unidad 2

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:
Gutiérrez Cepeda Andrés - 23211009

NOMBRE DEL MAESTRO (A):
HUMBERTO RAIZ OROZCO

9 de marzo 2026

1. Introducción

El objetivo de esta práctica es aplicar el Patrón Prototipo (Prototype) para optimizar la creación de objetos en C#. Específicamente, se desarrolló un sistema para un Instituto Tecnológico donde los docentes aplican exámenes a diferentes grupos a nivel de memoria y procesamiento, especialmente cuando estos objetos requieren configuraciones complejas o acceso a bases de datos.

Para resolver este problema, el diseño de software nos ofrece el Patrón Prototipo (Prototype), un patrón de tipo creacional que nos permite copiar o "clonar" objetos existentes en lugar de crearlos nuevamente. En esta práctica, se aplica este patrón al contexto de un Instituto Tecnológico, donde los docentes aplican exámenes a diferentes grupos. En lugar de generar un examen nuevo para cada grupo lo cual implicaría repetir el ingreso de datos como la clave de la materia, el nombre y el formato, se crea un "exámen base" por materia. Este prototipo se clona y únicamente se modifican los atributos que varían, como el identificador del grupo, el salón asignado o el docente titular, lo que evita reescribir datos repetitivos y reduce la creación innecesaria de nuevos objetos desde cero

2.Documentación Código}

La Interfaz y el Prototipo Abstracto (ICloneable y Examen)
La base del patrón reside en la clase abstracta Examen, la cual implementa la interfaz nativa de .NET llamada ICloneable. La magia de la clonación ocurre dentro del método Clone() al invocar this.MemberwiseClone(), lo que realiza una clonación superficial (Shallow Copy) del objeto en memoria.

```csharp
public abstract class Examen : ICloneable
{
    // Datos protegidos
    protected string claveMateria;
    protected string nombreAsignatura;
    protected string docente;
    protected string salon;
    protected string grupo;

    public Examen(string claveMateria, string nombreAsignatura, string docente, string salon)
    {
        this.claveMateria = claveMateria;
        this.nombreAsignatura = nombreAsignatura;
        this.docente = docente;
        this.salon = salon;
    }

    // Setters para los datos que varían al clonar
    public void SetGrupo(string grupo) { this.grupo = grupo; }
    public void SetDocente(string docente) { this.docente = docente; }
    public void SetSalon(string salon) { this.salon = salon; }

    // Método de clonación propio de C# (Shallow Copy)
    public object Clone()
    {
        return this.MemberwiseClone();
    }

    public abstract void MostrarExamen();
}
```

Los Prototipos Concretos (ExamenTeorico y ExamenPractico)

Mediante la herencia, estas clases extienden la clase base Examen y añaden atributos específicos. Al clonar uno de estos objetos, C# conserva automáticamente su tipo correcto (teórico o práctico) junto con sus atributos base.

```
public class ExamenTeorico : Examen
{
    protected int cantidadPreguntas;

    // Usamos 'base' en lugar de 'super' como se hace en Java
    public ExamenTeorico(string clave, string nombre, string docente, string salon, int preguntas)
        : base(clave, nombre, docente, salon)
    {
        this.cantidadPreguntas = preguntas;
    }

    public override void MostrarExamen()
    {
        // Uso de interpolación de strings para mayor legibilidad
        Console.WriteLine($"[TEÓRICO] {claveMateria} - {nombreAsignatura} | Docente: {docente} | Grupo: {grupo} | Salón:
    }
}
```

El Gestor o Catálogo (GestorExamenes)

Para no tener los prototipos flotando en el código, se utiliza un registro utilizando la colección Dictionary<string, Examen>. Cuando el programa principal necesita un examen, invoca ObtenerExamen("Clave"), el cual busca el prototipo y devuelve directamente la ejecución de .Clone().

```
public class GestorExamenes
{
    // En C# usamos Dictionary en lugar de HashMap
    private Dictionary<string, Examen> catalogoExamenes = new Dictionary<string, Examen>();

    public GestorExamenes()
    {
        CargarPrototiposBase();
    }

    private void CargarPrototiposBase()
    {
        catalogoExamenes.Add("Patrones", new ExamenPractico("SCD-1015", "Patrones de Diseño", "Ing. Pérez", "Lab 1", "Vi
        catalogoExamenes.Add("BasesDatos", new ExamenPractico("AEB-1011", "Bases de Datos", "Dra. Gómez", "Lab 2", "SQL
        catalogoExamenes.Add("Redes", new ExamenPractico("SCD-1021", "Redes de Computadoras", "Ing. López", "Lab Cisco",
        catalogoExamenes.Add("SistemasOperativos", new ExamenTeorico("AEC-1061", "Sistemas Operativos", "Mtro. Ramírez",
        catalogoExamenes.Add("IA", new ExamenPractico("SCC-1013", "Inteligencia Artificial", "Dra. Gómez", "Lab 3", "Pytl
        catalogoExamenes.Add("Calculo", new ExamenTeorico("ACF-0904", "Cálculo Vectorial", "Ing. Martínez", "Aula B2", 1
        catalogoExamenes.Add("Fisica", new ExamenTeorico("ACF-0902", "Física General", "Mtro. Ramírez", "Aula C3", 20));
        catalogoExamenes.Add("Estructuras", new ExamenPractico("AED-1026", "Estructura de Datos", "Ing. Pérez", "Lab 1",
    }

    public Examen ObtenerExamen(string claveBase)
    {
        if (catalogoExamenes.TryGetValue(claveBase, out Examen prototipo))
        {
            // Casteamos el object devuelto por Clone() a Examen
            return (Examen)prototipo.Clone();
        }
        return null;
```

3.Ejecución

```
--- GENERACIÓN DE EXÁMENES MEDIANTE CLONACIÓN (C#) ---

[PRÁCTICO] SCD-1015 - Patrones de Diseño | Docente: Ing. Pérez | Grupo: 7A | Salón: Lab 1 | Software: Visual Studio / C#
[PRÁCTICO] SCD-1015 - Patrones de Diseño | Docente: Ing. Pérez | Grupo: 7B | Salón: Lab 2 | Software: Visual Studio / C#

[PRÁCTICO] SCD-1021 - Redes de Computadoras | Docente: Ing. López | Grupo: 6A | Salón: Lab Cisco | Software: Packet Tracer
[PRÁCTICO] SCD-1021 - Redes de Computadoras | Docente: Ing. Ruiz | Grupo: 6B | Salón: Aula E4 | Software: Packet Tracer

[TEÓRICO] AEC-1061 - Sistemas Operativos | Docente: Mtro. Ramírez | Grupo: 5A | Salón: Aula A1 | Preguntas: 50
[PRÁCTICO] SCC-1013 - Inteligencia Artificial | Docente: Dra. Gómez | Grupo: 8A | Salón: Lab 3 | Software: Python / ML.NET
[TEÓRICO] ACF-0904 - Cálculo Vectorial | Docente: Ing. Martínez | Grupo: 2B | Salón: Aula B2 | Preguntas: 15
[TEÓRICO] ACF-0902 - Física General | Docente: Mtro. Ramírez | Grupo: 3C | Salón: Aula C3 | Preguntas: 20
[PRÁCTICO] AED-1026 - Estructura de Datos | Docente: Ing. Pérez | Grupo: 4A | Salón: Lab 1 | Software: C++ Compiler
[PRÁCTICO] AEB-1011 - Bases de Datos | Docente: Dra. Gómez | Grupo: 5C | Salón: Lab 2 | Software: SQL Server
>
```

4.Conclusión

Con esta práctica comprobé la utilidad del patrón Prototype para evitar la instancia repetitiva con new. En lugar de crear un examen desde cero para cada grupo, lo cual repite código innecesariamente, usar la interfaz ICloneable me permitió duplicar una plantilla base

Al aplicar la clonación con la interfaz ICloneable de C#, me di cuenta de lo eficiente que es tener una plantilla o prototipo base cargado en un gestor (diccionario). Me resultó muy interesante cómo puedo clonar un examen de Sistemas Operativos y simplemente usar un método SetGrupo() para adaptarlo, conservando intactos la clave de la materia y el nombre. En conclusión, este patrón me ha enseñado a pensar en la reutilización de objetos en memoria y En resumen, aplicar un gestor con un diccionario y clonar los objetos mediante MemberwiseClone() resultó ser una solución práctica para reutilizar atributos estáticos (como la clave y nombre de la materia) y solo modificar los datos dinámicos mediante métodos Set, como el grupo o el salón.